

Chapter 4 — Loops

AI Study Assistant — RAG Knowledge Base

Learning Objectives

Understand repetition in Python.

Use for and while loops.

Work with range().

Use break and continue appropriately.

1. Why Loops?

Loops repeat a block of code. They reduce duplicated code and are useful when an operation must be performed for multiple values.

2. for Loop

A for loop iterates over items in an iterable such as a list, string, tuple, or range.

```
for number in [1, 2, 3]:  
    print(number)
```

3. range()

range() commonly generates a sequence of integers for use in loops.

range(5) produces 0, 1, 2, 3, 4.

range(2, 6) produces 2, 3, 4, 5.

range(1, 10, 2) produces 1, 3, 5, 7, 9.

4. while Loop

A while loop repeats as long as its condition remains true.

```
count = 1  
while count <= 3:  
    print(count)  
    count += 1
```

The loop must make progress toward a condition becoming false to avoid unintended infinite loops.

5. break

break immediately exits the nearest loop.

```
for n in range(10):  
    if n == 5:  
        break  
    print(n)
```

6. continue

continue skips the rest of the current iteration and moves to the next iteration.

```
for n in range(5):  
    if n == 2:  
        continue  
    print(n)
```

7. Nested Loops

A loop can be placed inside another loop. The inner loop completes its iterations for each iteration of the outer loop.

Key Points for Revision

for is commonly used to iterate over an iterable.

while repeats while a condition is true.

range() is useful for generating integer sequences.

break exits a loop.

continue skips to the next iteration.